NMAM Institute of Technology, Nitte

Department of Computer Science and Engineering

Computer Networks Laboratory

LABORATORY MANUAL

Academic Year 2026–27

Name	: Prajwal Jogi
USN	:
Semester / Section	:

INDEX

Expt. No.	Date	Title of the Experiment	Page No.	Signature
1	14/08/2026	Point-to-point network: packet drops	1	
2	21/08/2026	TCP and UDP traffic: packet counts and throughput	3	
3	28/08/2026	Ethernet LAN: throughput versus data rate and error rate	6	
4	04/09/2026	Ethernet LAN: congestion window of TCP Vegas and Reno	8	
5	11/09/2026	Bit stuffing	10	
6	09/10/2026	Error detection using CRC (CRC-CCITT)	11	
7	07/10/2026	Frame sorting technique used in buffers	13	
8		File transfer using TCP sockets	14	
9		Calculator using UDP sockets	16	
10	06/11/2026	Distance vector routing algorithm	18	

Experiment No. 1

Point-to-point network: packet drops

Aim: Simulate a three-node point-to-point network with duplex links between them. Set the queue size, vary the bandwidth and find the number of packets dropped.

Program: exp1.tcl

```
# gedit exp1.tcl
set ns [new Simulator]

# open a new file for NAMTRACE
set nf [open out.nam w]
$ns namtrace-all $nf

# open a new file to log TRACE
set tf [open out.tr w]
$ns trace-all $tf

# body of finish procedure
proc finish {} {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
    exec nam out.nam &
    exit 0
}

# Create nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]

# Create links between nodes
$ns duplex-link $n0 $n1 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail

# set orientation
$ns duplex-link-op $n0 $n1 orient right-up
$ns duplex-link-op $n1 $n2 orient right-down

# set queue limit-default is 50 packets
$ns queue-limit $n0 $n1 50
$ns queue-limit $n1 $n2 50

# Create transport agent
# Sender
set udp0 [new Agent/UDP]
$ns attach-agent $n0 $udp0

# Receiver
set null0 [new Agent/Null]
$ns attach-agent $n2 $null0
$ns connect $udp0 $null0

# Create Application to generate traffic
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0

# Start and stop generating traffic
$ns at 0.5 "$cbr0 start"
$ns at 4.5 "$cbr0 stop"

$ns at 5.0 "finish"

$ns run
```

Program: exp1.awk

```
# gedit exp1.awk
BEGIN { count = 0; }
{
    if( $1 == "d")
    {
        count++;
        printf("%s\t %s\n", $5 , $11 );
    }
}
END { printf("The number of packets dropped =%d \n", count ); }
```

Output

```
$ ns exp1.tcl
$ awk -f exp1.awk out.tr
 The number of packets dropped =0

$ awk -f exp1.awk out.tr        # bandwidth changed to 0.5Mb
cbr      131
cbr      134
cbr      136
...
 The number of packets dropped =252
```

Experiment No. 2

TCP and UDP traffic: packet counts and throughput

Aim: Simulate a four-node point-to-point network and connect the links as follows: n0-n2, n1-n2 and n2-n3. Apply a TCP agent between n0-n3 and a UDP agent between n1-n3. Apply relevant applications over the TCP and UDP agents, change the parameters, determine the number of packets sent by TCP and UDP and analyse the throughput.

Program: exp2.tcl

```
set ns [new Simulator]

# open a new file for NAMTRACE
set nf [open out.nam w]
$ns namtrace-all $nf

# open a new file to log TRACE
set tf [open out.tr w]
$ns trace-all $tf

# body of finish procedure
proc finish {} {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
    exec nam out.nam &
    exit 0
}

# Create 4 nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]

# set label
$n0 label "TCP"
$n1 label "UDP"
$n2 label "Receiver"

# Create links between nodes
$ns duplex-link $n0 $n2 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
$ns duplex-link $n2 $n3 1Mb 10ms DropTail

# set orientation
$ns duplex-link-op $n0 $n2 orient right-down
$ns duplex-link-op $n1 $n2 orient right-up
$ns duplex-link-op $n2 $n3 orient right

# set the queue limit-default is 50 packets
$ns queue-limit $n0 $n2 50
$ns queue-limit $n1 $n2 50
$ns queue-limit $n2 $n3 50

# Create TCP agent b/w node 0 and node 3
# Sender
set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0

# Receiver
set sink0 [new Agent/TCPSink]
$ns attach-agent $n3 $sink0
$ns connect $tcp0 $sink0

# Create FTP Application for TCP Agent
set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0

# Specify TCP packet size
Agent/TCP set packetSize_ 1000

```

```
# Create UDP Agent b/w node1 and node3
# Sender
set udp0 [new Agent/UDP]
$ns attach-agent $n1 $udp0

# Receiver
set null0 [new Agent/Null]
$ns attach-agent $n3 $null0
$ns connect $udp0 $null0

# Create CBR Application for UDP Agent
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0

# Start and Stop FTP Traffic
$ns at 0.75 "$ftp0 start"
$ns at 4.75 "$ftp0 stop"

# Start and stop CBR traffic
$ns at 0.5 "$cbr0 start"
$ns at 4.5 "$cbr0 stop"

# Stop the simulations
$ns at 5.0 "finish"
$ns run
```

Program: exp2.awk

```
# gedit exp2.awk
BEGIN {
    tcp_count = 0;
    udp_count = 0;
}
{
    if( $5 == "cbr")
        udp_count++;
    else if( $5 == "tcp")
        tcp_count++;
}
END {
    printf("The number of packets by TCP =%d\n", tcp_count );
    printf("The number of packets by UDP =%d\n", udp_count );
}
```

Program: exp2_throughput.awk

```
# gedit exp2_throughput.awk
BEGIN {
    sSize = 0;
    startTime = 5.0;
    stopTime = 0.1;
    Tput = 0;
}
{
    event = $1;
    time = $2;
    size = $6;
    if( event == "+")
    {
        if ( time < startTime ) {
            startTime = time;
        }
    }
    if( event == "r")
    {
        if( time > stopTime )
        {
            stopTime = time;
        }
        sSize += size;
```

```
    }
    Tput = ( sSize / ( stopTime - startTime )) * (8/1000) ;
    printf("%f\t %.2f\n", time , Tput );
}
END {
}
```

Output

```
$ ns exp2.tcl
$ awk -f exp2.awk out.tr
The number of packets by TCP =846
The number of packets by UDP =4786

$ awk -f exp2_throughput.awk out.tr > plot_throughput
$ xgraph plot_throughput
$ head -3 plot_throughput; tail -2 plot_throughput
0.500000         -0.00
0.500000         -0.00
0.505000         -0.00
...
4.800920         2017.06
4.809240         2013.24
```

Experiment No. 3

Ethernet LAN: throughput versus data rate and error rate

Aim: Simulate an Ethernet LAN using N nodes (6-10). Change the error rate and the data rate and compare the throughput.

Program: exp3.tcl

```
set ns [new Simulator]
set namfile [open exp3.nam w]
$ns namtrace-all $namfile
set tracefile [open exp3.tr w]
$ns trace-all $tracefile

proc finish {} {
    global ns namfile tracefile
    $ns flush-trace
    close $namfile
    close $tracefile
    exec nam exp3.nam &
    exec awk -f exp3.awk exp3.tr > exp3.graph &
    exit 0
}

# Create 6 nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
set n6 [$ns node]

# label source and sink nodes
$n0 label "TCPSource"
$n6 label "TCPSink"

# color to the flow
$ns color 1 red

# color to the nodes
$n0 color blue
$n6 color orange

$ns duplex-link $n0 $n1 3Mb 20ms DropTail
$ns make-lan "$n1 $n2 $n3 $n4 $n5 $n6" 2Mb 40ms LL Queue/DropTail Mac/802_3

$ns duplex-link-op $n0 $n1 orient right
set tcp [new Agent/TCP]
$ns attach-agent $n0 $tcp
$tcp set class_ 1

set tcpsink [new Agent/TCPSink]
$ns attach-agent $n6 $tcpsink
$ns connect $tcp $tcpsink

set ftp [new Application/FTP]
$ftp attach-agent $tcp

$ns at 1.0 "$ftp start"
$ns at 5.0 "$ftp stop"
$ns at 5.5 "finish"
$ns run
```

Lines added to exp3.tcl to introduce an error rate

```
# Error model on the link n0 -> n1
set em [new ErrorModel]
$em unit pkt
$em set rate_ 0.01
$em ranvar [new RandomVariable/Uniform]
$em drop-target [new Agent/Null]
```

```
$ns lossmodel $em $n0 $n1
```

Program: exp3.awk

```
# gedit exp3.awk
BEGIN {
    sSize = 0;
    startTime = 5.0;
    stopTime = 0.1;
    Tput = 0;
}
{
    event = $1;
    time = $2;
    size = $6;
    if ( event == "+")
    {
        if ( time < startTime )
        {
            startTime = time;
        }
    }
    if ( event == "r")
    {
        if ( time > stopTime )
        {
            stopTime = time;
        }
        sSize += size;
    }
    Tput = ( sSize /( stopTime - startTime )) * (8/1000) ;
    printf("%f\t %.2f\n", time , Tput );
}
END {
}
```

Output

```
$ ns exp3.tcl
$ xgraph exp3.graph
$ head -3 exp3.graph; tail -2 exp3.graph
1.000000          -0.00
1.000000          -0.00
1.020107         15.91
...
5.202373         1328.39
5.206637         1327.12
```

Experiment No. 4

Ethernet LAN: congestion window of TCP Vegas and Reno

Aim: Simulate an Ethernet LAN using N nodes and set multiple traffic nodes, and determine the collisions across the different nodes. Also plot the congestion window for different source/destination pairs.

Program: exp4.tcl

```
set ns [new Simulator]
set namfile [open exp4.nam w]
$ns namtrace-all $namfile

set tracefile [open exp4.tr w]
$ns trace-all $tracefile

proc finish {} {
    global ns namfile tracefile tfile1 tfile2
    $ns flush-trace
    close $namfile
    close $tracefile
    close $tfile1
    close $tfile2
    exec nam exp4.nam &
    exec awk -f exp4.awk cwnd1.tr > tcp1 &
    exec awk -f exp4.awk cwnd2.tr > tcp2 &
    exit 0
}

# Create the nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
set n6 [$ns node]
set n7 [$ns node]
set n8 [$ns node]

# color to packet flow
$ns color 1 Blue
$ns color 2 Red

# set shape and color to node 7 and node 8
$n7 shape Box
$n7 color Blue
$n8 shape hexagon
$n8 color Red

$ns duplex-link $n1 $n0 2Mb 10ms DropTail
$ns duplex-link $n2 $n0 2Mb 10ms DropTail
$ns duplex-link $n0 $n3 1Mb 20ms DropTail

# Create LAN
$ns make-lan "$n3 $n4 $n5 $n6 $n7 $n8" 512Kb 40ms LL Queue/DropTail Mac/802_3

# Set Orientation
$ns duplex-link-op $n1 $n0 orient right-down
$ns duplex-link-op $n2 $n0 orient right-up
$ns duplex-link-op $n0 $n3 orient right
$ns queue-limit $n0 $n3 20

# set the transport protocol
set tcp1 [new Agent/TCP/Vegas]
$ns attach-agent $n1 $tcp1

set sink1 [new Agent/TCPSink]
$ns attach-agent $n7 $sink1

# set the application
set ftp1 [new Application/FTP]
$ftp1 attach-agent $tcp1

```

```
$ns connect $tcp1 $sink1
$tcp1 set class_ 1
$tcp1 set packetSize_ 55

set tfile1 [open cwnd1.tr w]
$tcp1 attach $tfile1
$tcp1 trace cwnd_

set tcp2 [new Agent/TCP/Reno]
$ns attach-agent $n2 $tcp2

set sink2 [new Agent/TCPSink]
$ns attach-agent $n8 $sink2

set ftp2 [new Application/FTP]
$ftp2 attach-agent $tcp2

$ns connect $tcp2 $sink2
$tcp2 set class_ 2
$tcp2 set packetSize_ 55

set tfile2 [open cwnd2.tr w]
$tcp2 attach $tfile2
$tcp2 trace cwnd_

$ns at 0.5 "$ftp1 start"
$ns at 1.0 "$ftp2 start"
$ns at 5.0 "$ftp2 stop"
$ns at 5.0 "$ftp1 stop"
$ns at 5.5 "finish"
$ns run
```

Program: exp4.awk

```
# exp4.awk
BEGIN {
}
{
    if( $6 == "cwnd_")
    {
        printf("%f\t %f\n", $1 , $7 );
    }
}
END {
}
```

Output

```
$ ns exp4.tcl
$ xgraph tcp1
$ xgraph tcp2
$ head -4 tcp1; tail -2 tcp1
0.000000         1.000000
0.946450         2.000000
0.946450         3.000000
1.392900         4.000000
...
5.483430         1.000000
5.483430         2.000000
```

Experiment No. 5

Bit stuffing

Aim: Write a program to implement the bit stuffing (or byte stuffing) concept in the data-link layer.

Program: exp5.py

```
def bit_stuffing(data):
    stuffed_data = ""
    count = 0
    for bit in data:
        stuffed_data += bit
        if bit == '1':
            count += 1
            if count == 5:
                stuffed_data += '0'
                count = 0
        else:
            count = 0
    return stuffed_data

def bit_destuffing(stuffed_data):
    destuffed_data = ""
    count = 0
    skip_next = False
    for bit in stuffed_data:
        if skip_next:
            skip_next = False
            continue
        destuffed_data += bit
        if bit == '1':
            count += 1
            if count == 5:
                skip_next = True
                count = 0
        else:
            count = 0
    return destuffed_data

data = input("Enter the input data :")
print("Original data :", data)
stuffed = bit_stuffing(data)
print("Bit Stuffed data :", stuffed)
destuffed = bit_destuffing(stuffed)
print("Bit Destuffed data :", destuffed)
```

Output

```
$ python3 exp5.py
Enter the input data :1011111101
Original data : 1011111101
Bit Stuffed data : 10111110101
Bit Destuffed data : 1011111101
```

Experiment No. 6

Error detection using CRC (CRC-CCITT)

Aim: Write a program for an error-detecting code using CRC-CCITT (16-bit).

Program: exp6.py

```
div = [1, 0, 1, 1]
cdata = []
n = int(input("Enter the number of bits :"))
data = [int(input(f"Enter bit {i + 1}: ")) for i in range(n)]
m = len(div)

if n < m:
    print("Invalid data ")
else:
    cdata = data.copy()
    for i in range(m - 1):
        cdata.append(0)
    print("Initial Codeword :", cdata)
    for i in range(n):
        if cdata[i] == 1:
            for j in range(m):
                cdata[i + j] ^= div[j]

    for i in range(n, n + m - 1):
        data.append(cdata[i])
    print("Codeword to be transmitted is:", data)
    ch = input("Introduce Error - Y -> YES N->NO:")
    if ch.upper() == 'Y':
        p = int(input("Enter the position to change bit :"))
        if 1 <= p <= len(data):
            data[p-1] ^= 1
        else:
            print("Invalid Position !")
    print("The message received is:", data)
    for i in range(n):
        if data[i] == 1:
            for j in range(m):
                data[i + j] ^= div[j]
    flag = True
    for i in data:
        if i == 1:
            flag = False
            break
    if (flag):
        print("No Error \nSuccessfully Transferred")
    else:
        print("Error !")
```

Output

Run 1 - no error

```
$ python3 exp6.py
Enter the number of bits :4
Enter bit 1: 1
Enter bit 2: 0
Enter bit 3: 0
Enter bit 4: 1
Initial Codeword : [1, 0, 0, 1, 0, 0, 0]
Codeword to be transmitted is: [1, 0, 0, 1, 1, 1, 0]
Introduce Error - Y -> YES N->NO:N
The message received is: [1, 0, 0, 1, 1, 1, 0]
No Error
Successfully Transferred
```

Run 2 - error introduced

```
$ python3 exp6.py
Enter the number of bits :4
Enter bit 1: 1
Enter bit 2: 0
Enter bit 3: 0
Enter bit 4: 1
Initial Codeword : [1, 0, 0, 1, 0, 0, 0]
Codeword to be transmitted is: [1, 0, 0, 1, 1, 1, 0]
Introduce Error - Y -> YES N->NO:Y
Enter the position to change bit :1
The message received is: [0, 0, 0, 1, 1, 1, 0]
Error !
```

Experiment No. 7

Frame sorting technique used in buffers

Aim: Write a program for the frame sorting technique used in buffers.

Program: exp7.py

```
import random
DATA_SZ = 3

def bubb_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

msg = input("Enter the message:")
msg_chunks = [msg[i:i + DATA_SZ] for i in range(0, len(msg), DATA_SZ)]
frames = list(enumerate(msg_chunks, start=1))
print("Fragmented frames:", frames)
for i in range(len(frames)):
    j = random.randint(0, len(frames) - 1)
    frames[i], frames[j] = frames[j], frames[i]
print("Shuffled frames:", frames)
bubb_sort(frames)
print("Sorted frames:", frames)
print("Sorted message:" + "".join(x[1] for x in frames))
```

Output

```
$ python3 exp7.py
Enter the message:NMAMIT NITTE
Fragmented frames: [(1, 'NMA'), (2, 'MIT'), (3, ' NI'), (4, 'TTE')]
Shuffled frames: [(2, 'MIT'), (3, ' NI'), (1, 'NMA'), (4, 'TTE')]
Sorted frames: [(1, 'NMA'), (2, 'MIT'), (3, ' NI'), (4, 'TTE')]
Sorted message:NMAMIT NITTE
```

Experiment No. 8

File transfer using TCP sockets

Aim: Using TCP/IP sockets, write a client-server program in which the client sends the name of a file and the server sends back the contents of the requested file if it is present.

Program: tcp_server.py

```
# tcp_server.py - sends the requested file to the client
import socket
import os

HOST = "127.0.0.1"               # this computer only (localhost)
PORT = 8888                      # port number the server listens on
BUFFER_SIZE = 1024               # maximum bytes received by one recv()

# create a TCP socket, bind it to (HOST, PORT) and start listening
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_socket.bind((HOST, PORT))
server_socket.listen(1)
print("Server is listening on", HOST, "port", PORT)

try:
    while True:
        client_socket, client_address = server_socket.accept()
        with client_socket:
            print("Accepted connection from", client_address)

            # receive the file name sent by the client
            file_name = client_socket.recv(BUFFER_SIZE).decode()
            print("Received file name =", file_name)

            # send the contents if the file exists, otherwise an error text
            if os.path.isfile(file_name):
                with open(file_name, "rb") as file:
                    client_socket.sendall(file.read())
                print("File sent")
            else:
                client_socket.sendall(b"File not found")
                print("File not found")
except KeyboardInterrupt:
    print("\nServer stopped")
finally:
    server_socket.close()
```

Program: tcp_client.py

```
# tcp_client.py - requests a file from the server and saves it
import socket
import sys
import os

SERVER_ADDRESS = ("127.0.0.1", 8888)    # (IP address, port) of the server
BUFFER_SIZE = 1024

# create a TCP socket and connect to the server
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
try:
    client_socket.connect(SERVER_ADDRESS)
except ConnectionRefusedError:
    sys.exit("Could not connect to the server. Is tcp_server.py running?")

file_name = input("Enter the file name: ").strip()
if file_name == "":
    client_socket.close()
    sys.exit("No file name entered.")

# send the file name, then read until the server closes the connection
client_socket.sendall(file_name.encode())
file_contents = b""
while True:
```

```
    chunk = client_socket.recv(BUFFER_SIZE)
    if not chunk:                       # empty bytes: the server has closed
        break
    file_contents += chunk
client_socket.close()

if file_contents == b"File not found":
    print("File not found on the server.")
else:
    print("File contents:")
    print(file_contents.decode(errors="replace"))
    save_name = "received_" + os.path.basename(file_name)
    with open(save_name, "wb") as file:
        file.write(file_contents)
    print("File received and saved as '" + save_name + "'")
```

Test file: sample.txt

```
Computer Networks Laboratory
Experiment 8: file transfer using TCP sockets
This file was sent by the server.
```

Output

Terminal 1 - server

```
$ python3 tcp_server.py
Server is listening on 127.0.0.1 port 8888
Accepted connection from ('127.0.0.1', 48040)
Received file name = sample.txt
File sent
Accepted connection from ('127.0.0.1', 48042)
Received file name = missing.txt
File not found
^C
Server stopped
```

Terminal 2 - client

```
$ python3 tcp_client.py
Enter the file name: sample.txt
File contents:
Computer Networks Laboratory
Experiment 8: file transfer using TCP sockets
This file was sent by the server.

File received and saved as 'received_sample.txt'
$ python3 tcp_client.py
Enter the file name: missing.txt
File not found on the server.
```

Experiment No. 9

Calculator using UDP sockets

Aim: Using UDP sockets, write a client-server program in which the client sends two numbers and an operator and the server responds with the result. Display the result and appropriate messages for invalid inputs at the client side.

Program: udp_server.py

```
# udp_server.py - receives "number operator number" and replies with the result
import socket

SERVER_IP = "127.0.0.1"
SERVER_PORT = 20001
BUFFER_SIZE = 1024

# create a UDP socket and bind it to (SERVER_IP, SERVER_PORT)
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_socket.bind((SERVER_IP, SERVER_PORT))
print("UDP server up and listening on port", SERVER_PORT)

try:
    while True:
        # wait for a datagram; recvfrom() also tells us who sent it
        data, client_address = server_socket.recvfrom(BUFFER_SIZE)
        request = data.decode(errors="replace")
        print("Request from", client_address, ":", request)

        # work out the result (or an error message)
        try:
            num1, operator, num2 = request.split()
            a = float(num1)
            b = float(num2)
            if operator == "+":
                result = a + b
            elif operator == "-":
                result = a - b
            elif operator == "*":
                result = a * b
            elif operator == "/":
                if b == 0:
                    result = "Error: division by zero"
                else:
                    result = a / b
            elif operator == "%":
                if b == 0:
                    result = "Error: modulo by zero"
                else:
                    result = a % b
            else:
                result = "Error: invalid operator"
        except ValueError:
            result = "Error: invalid request"

        print("Result to client:", result)
        server_socket.sendto(str(result).encode(), client_address)
except KeyboardInterrupt:
    print("\nServer stopped")
finally:
    server_socket.close()
```

Program: udp_client.py

```
# udp_client.py - reads two numbers and an operator, asks the server to calculate
import socket
import sys
import math

SERVER_ADDRESS = ("127.0.0.1", 20001)   # (IP address, port) of the server
BUFFER_SIZE = 1024

def is_number(text):
    # True only for finite numbers such as 12, -3.5 or 1e3
```

```
    try:
        return math.isfinite(float(text))
    except ValueError:
        return False

# read and check the input before anything is sent
num1 = input("Enter first number : ").strip()
operator = input("Enter operator (+ - * / %) : ").strip()
num2 = input("Enter second number : ").strip()

if not is_number(num1) or not is_number(num2):
    sys.exit("Invalid input: both operands must be numbers.")
if operator not in ("+", "-", "*", "/", "%"):
    sys.exit("Invalid operator: use one of + - * / %")

# create a UDP socket; wait at most 5 seconds for the reply
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client_socket.settimeout(5)

request = num1 + " " + operator + " " + num2
client_socket.sendto(request.encode(), SERVER_ADDRESS)

try:
    reply, server_address = client_socket.recvfrom(BUFFER_SIZE)
    print(request, "=", reply.decode())
except (socket.timeout, ConnectionResetError):
    print("No reply from the server. Is udp_server.py running?")
finally:
    client_socket.close()
```

Output

Terminal 1 - server

```
$ python3 udp_server.py
UDP server up and listening on port 20001
Request from ('127.0.0.1', 42284) : 12 + 5
Result to client: 17.0
Request from ('127.0.0.1', 42871) : 20 / 4
Result to client: 5.0
Request from ('127.0.0.1', 59562) : 5 / 0
Result to client: Error: division by zero
^C
Server stopped
```

Terminal 2 - client

```
$ python3 udp_client.py
Enter first number : 12
Enter operator (+ - * / %) : +
Enter second number : 5
12 + 5 = 17.0
$ python3 udp_client.py
Enter first number : 20
Enter operator (+ - * / %) : /
Enter second number : 4
20 / 4 = 5.0
$ python3 udp_client.py
Enter first number : 5
Enter operator (+ - * / %) : /
Enter second number : 0
5 / 0 = Error: division by zero
$ python3 udp_client.py
Enter first number : abc
Enter operator (+ - * / %) : +
Enter second number : 5
Invalid input: both operands must be numbers.
```

Experiment No. 10

Distance vector routing algorithm

Aim: Write a program in C for the distance vector routing algorithm to find a suitable path for transmission, and display the routing table of every router.

Program: exp10.c

```c
#include <stdio.h>
#include <stdlib.h>
struct node
{
    int dist[20];
    int from[20];
} rt[10];
int main()
{
    int dmat[20][20];
    int n,i,j,k,c=1;
    printf("Enter the number of nodes: \n");
    scanf("%d", &n);
    printf("Enter the cost matrix: ");
    for(int i=1; i<=n; i++)
    {
        for(int j=1; j<=n; j++)
        {
            scanf("%d", &dmat[i][j]);
            dmat[i][i]=0;
            rt[i].dist[j]=dmat[i][j];
            rt[i].from[j]=j;
        }
    }
    do
    {
        for(i=1; i<=n; i++)
            for(j=1; j<=n; j++)
                for(k=1; k<=n; k++)
                    if(rt[i].dist[j]>dmat[i][k] + rt[k].dist[j])
                    {
                        rt[i].dist[j]=rt[i].dist[k]+rt[k].dist[j];
                        rt[i].from[j]=k;
                    }
        c++;
    }
    while(c<n);
    for(i=1; i<=n; i++)
    {
        printf("\nDistance Table for router %c is in \n",i+64);
        for(j=1;j<=n;j++)
            printf("\tNode %d Via %d.  Distance: %d.\n",j,rt[i].from[j], rt[i].dist[j]);
    }
    return 0;
}
```

Output

Run 1 - three routers (recorded in the laboratory)

```
$ gcc exp10.c -o exp10
$ ./exp10
Enter the number of nodes:
3
Enter the cost matrix: 1 0 1
2 5 3
3 1 0

Distance Table for router A is in
        Node 1 Via 1.  Distance: 0.
        Node 2 Via 2.  Distance: 0.
        Node 3 Via 3.  Distance: 1.

Distance Table for router B is in
        Node 1 Via 1.  Distance: 2.
```

```
        Node 2 Via 2.  Distance: 0.
        Node 3 Via 3.  Distance: 3.

Distance Table for router C is in
        Node 1 Via 1.  Distance: 3.
        Node 2 Via 2.  Distance: 1.
        Node 3 Via 3.  Distance: 0.
```

Run 2 - four routers (no direct link A-C, entered as 999)

```
$ ./exp10
Enter the number of nodes:
4
Enter the cost matrix: 0 2 999 1
2 0 3 7
999 3 0 11
1 7 11 0

Distance Table for router A is in
        Node 1 Via 1.  Distance: 0.
        Node 2 Via 2.  Distance: 2.
        Node 3 Via 2.  Distance: 5.
        Node 4 Via 4.  Distance: 1.

Distance Table for router B is in
        Node 1 Via 1.  Distance: 2.
        Node 2 Via 2.  Distance: 0.
        Node 3 Via 3.  Distance: 3.
        Node 4 Via 1.  Distance: 3.

Distance Table for router C is in
        Node 1 Via 2.  Distance: 5.
        Node 2 Via 2.  Distance: 3.
        Node 3 Via 3.  Distance: 0.
        Node 4 Via 2.  Distance: 6.

Distance Table for router D is in
        Node 1 Via 1.  Distance: 1.
        Node 2 Via 1.  Distance: 3.
        Node 3 Via 1.  Distance: 6.
        Node 4 Via 4.  Distance: 0.
```